

仓颉生态创新开发挑战赛

regex4cj 作品提交说明

基础信息（必填）

赛事作品名称	regex4cj		
队伍名称	Re:cj		
队伍负责人	闫家梁		
队伍成员	闫家梁，刘孝凡		
联系电话	13453426633	联系邮箱	3149269490@qq.com
作品类型	☒ 通用工具库		
使用环境	☒ 仓颉原生环境 已验证：STS 仓颉 1.1.3 / Apple Silicon macOS。		

一、项目概述

1. 项目一句话定位：regex4cj 是面向仓颉开发者的原生正则表达式库，提供文本与字节匹配、捕获、替换、拆分及多规则分类，并以可复核测试验证其与原仓库的行为一致性。

2. 项目开发背景：正则表达式广泛用于日志处理、字段校验和文本清洗。将成熟库从 Rust 转为仓颉，不只是改写语法，还需要处理 UTF-8 字节位置、空匹配、分组捕获、匹配优先级等细节。本项目选择 rust-lang/regex 为参考，建立仓颉实现和对照验证流程，降低业务接入和后续维护成本。

3. 项目核心目标：形成可独立获取、构建、调用和测试的仓颉库；优先保证常用功能可用、行为边界清晰、结果可复现。当前版本不宣称已实现原仓库全部接口，也不以未经定义的覆盖百分比代表完成度。

二、需求分析与适用人群

1. 目标用户精准定位：需要在仓颉程序中校验或处理文本的应用开发者；需要批量识别日志、订单等内容的工具开发者；学习正则语法、自动机和跨语言移植的开发者。
2. 行业开发痛点分析：手写字符串处理容易遗漏边界；多条规则的分类与捕获需求难以统一；跨语言移植中“编译通过”不等于“返回结果相同”。中文与混合编码文本、零长度匹配和重复分组尤其容易产生隐蔽差异。
3. 库解决方案：用统一 API 封装编译、查找、捕获和变换；用 `RegexSet` 返回命中的规则编号；字符串 API 使用 UTF-8 字节偏移，字节 API 支持任意字节；用固定版本 Rust 参考程序及仓库内测试数据核对结果。

三、作品核心能力与功能设计

1. 基础通用能力（作品核心必备功能，保障可用、好用）

（1）能力一：文本和字节正则处理

功能介绍：Regex、BytesRegex 提供匹配判断、查找、分组捕获、替换、拆分和指定起点搜索，并支持惰性迭代。

适用开发场景：六位数字校验、订单号提取、日志清洗和字段脱敏。

调用方式简述：构造 `Regex("[0-9]{6}")` 后调用 `isMatch("123456")`，得到 `true`。

实现效果与价值：业务代码直接调用库接口，无需自行实现匹配算法；返回位置的单位明确，便于截取原始文本。

(2) 能力二：多规则分类与语法、引擎接口

功能介绍：RegexSet、BytesRegexSet 一次返回所有命中规则的编号；另提供部分 AST/HIR 语法结构、字面量提取和自动机接口。

适用开发场景：按订单、退款、发票规则分类文本；观察表达式结构和尝试不同匹配路径。

调用方式简述：预先构造规则集合，再查询文本得到编号集合；需要某条规则的捕获内容时，使用对应 Regex 查询。

实现效果与价值：将分类与内容提取分开，复用规则；语法和引擎接口适合教学与扩展，但其范围小于原仓库完整接口。

2. 创新特色能力 (赛事加分核心，重点填写)

(1) 创新点一：围绕行为建立移植验证闭环

创新详情：将仓颉结果与固定 Rust 版本的结果对照，覆盖捕获、起点、迭代等容易出错的细节；验收记录绑定输入文件哈希。

工程价值：区别于仅检查编译和少量示例，能够追踪行为回归。本项目不宣称发明了新的正则算法，也不宣称优于其他仓颉库。

(2) 创新点二：可独立复现的交付结构

创新详情：库代码、消费示例、测试数据、脚本、许可和验证记录集中在一个仓库；Rust 对照程序只服务于开发验证，不是运行时依赖。

工程价值：评审和使用者可按 README 配置环境，从统一命令入口运行演示、调用示例和测试。

四、仓颉语言核心特性应用

1. 应用仓颉特性 1：枚举与模式匹配

技术实现与封装逻辑：使用枚举描述表达式节点和指令类型，通过 match 分派处理解析与执行逻辑。

落地优势与效果：把不同节点的处理逻辑显式组织起来，使语法结构和执行分支更易检查与维护。

2. 应用仓颉特性 2：泛型集合与 Option 类型

技术实现与封装逻辑：使用标准集合管理状态和匹配结果，用 Option 表达可缺失的匹配或分组。

落地优势与效果：区分“未匹配”和“匹配到空串”，减少用特殊数字或空字符串表示多种含义的歧义。

3. 应用仓颉特性 3：包与静态类型接口

技术实现与封装逻辑：以 regex4cj 包导出库能力，通过 cjpm 配置供独立消费示例引用，结合仓颉测试设施验证调用。

落地优势与效果：业务程序可按依赖接入，编译阶段检查类型；库实现、命令行和测试职责分离。

开发技术栈与工具：STS 仓颉 1.1.3、cjpm、cjc、cjlint、cjfmt；Python 3.9+ 执行验证脚本，Rust/Cargo 用于完整差分验证。开发过程中使用 Codex、Grok 辅助分析和编写；本次不申报独立 AI 工具、Agent、MCP 或 RAG 成果。

五、库性能与兼容性设计

1. 性能优化设计：复用已编译表达式，使用惰性迭代减少一次性结果构造，并提供 DFA、稀疏状态和 Hybrid 缓存等执行能力。当前没有充分的跨平台性能基准，不承诺与 Rust 实现吞吐量或内存占用相同，也不报告未经测量的加速比例。

2. 兼容性设计：当前已验证环境为 Apple Silicon macOS、STS 仓颉 1.1.3。字符串结果使用 UTF-8 字节偏移，Unicode 数据版本为 16。运行库不调用 Rust，也无第三方包运行依赖；构建仍需正确安装仓颉 SDK 和系统工具。Linux、Windows、鸿蒙环境未完成验证。

3. 稳定性与容错设计：对非法表达式返回错误，按“功能—特性—输入与预期结果”组织 15 个功能组，检查空匹配、捕获优先级、UTF-8 偏移、迭代器复制和缓存复用等边界。最近一次完整验收中，47 个批量对照套件、77 项仓颉原生测试、91 个具名案例和 15 项 Python 工具测试全部通过。库代码可执行行覆盖为 10883/11671 (93.25%)，451 个公开可调用声明均被执行；声明数含构造方法及重载，不等于独立功能数量，代码覆盖率也不代表原仓库功能完成度。

六、使用说明与落地场景

1. 快速接入说明

按仓库根目录 README.md 安装并配置 STS 仓额 1.1.3 和 Python 3.9+，在仓库根目录执行：

```
bash scripts/run.sh demo
bash scripts/run.sh example
bash scripts/run.sh check
```

分别运行演示、独立消费示例和基础检查。完整差分验证执行 `bash scripts/run.sh verify`，另需 Rust/Cargo；首次获取锁定依赖可能需要联网。实测代码覆盖率执行 `bash scripts/run.sh coverage`。环境配置、结果位置和故障排查统一见 README.md。

其他仓额项目可在 `cjpm.toml` 中配置本地源码依赖：

```
[dependencies]
regex4cj = { path = "../regex4cj/port" }
```

程序通过 `import regex4cj.*` 使用 Regex 等接口。当前提供源码接入方式，尚未发布到中央仓。

2. 落地应用场景

字段校验：规则 `^[0-9]{6}$` 接受 123456，拒绝 12345 和 abc123。

中文提取：执行 `bash scripts/run.sh find '\p{Han}+' 'A 中文 α'`，匹配“中文”，位置为 [1,7)，单位是 UTF-8 字节，而非字符序号。

业务分类：对“订单 AB-123 需要退款，也需要开发票”执行 `classify` 演示，可得到相应三条规则的编号 [0,1,2]。以上是随仓库提供的演示场景，不代表已上线商业系统。

3. 核心价值

开发价值：用可复用接口完成常见文本任务，并提供可直接运行的示例。

生态价值：提供原生仓额实现、测试与许可说明，便于后续接入和维护。

创新价值：将 AI 辅助移植结果纳入可追溯验证，明确实现范围与未覆盖行为。

七、可行性与开发落地规划

1. 技术可行性：当前版本已完成本地构建、主功能演示和多层验证，13 个单模式演示和 7 个分类演示全部通过。验收记录保留受测文件的 SHA-256，便于核对实现与测试快照。具名案例同时核对人工预期和固定 Rust 版本结果；测试通过不代表任意表达式、

输入和配置均已穷举。

2. 落地可行性：已通过干净副本构建和独立消费示例验证。交付包含源码、固定测试数据、Unicode 数据快照及必要许可；SDK 和工具链由使用者安装。基础验证不需要 Rust，完整对照验证使用锁定的 Rust 依赖。当前验证平台为 Apple Silicon macOS，其他平台和中央仓安装仍需后续验证。

3. 开发进度规划：本次提交 0.3.0 版本，交付库源码、使用文档、示例、测试和验证记录。后续按实际需求补充跨平台验证、尚缺的语法和引擎契约及性能基准；这些方向不计入本次已完成功能。

八、作品总结与未来展望

1. 项目总结：项目交付了可构建、可调用、可演示、可验证的仓颉正则库。主要应用功能已有测试支撑，同时提供部分语法及自动机能力。它是对原仓库的移植与适配成果，并非原仓库整个工作区的完整等价替代。

2. 未来展望：优先保持已有行为稳定和文档准确；在有明确需求时扩展平台测试与原仓库契约覆盖，补齐独立执行引擎和序列化等能力，再以可重复基准评价性能。

九、仓颉 AI 生态贡献加分申请（可选）

本次不申请此项加分。项目使用 AI 辅助开发，但未提交独立可复用的仓颉 AI 生态工具成果；一般 AI 辅助开发不作为本项贡献申报。

申请情况（必选）：☒ 不申请仓颉 AI 生态贡献加分 ☐ 申请仓颉 AI 生态贡献加分

十、补充说明（可选）

代码与参考材料

项目仓库：<https://github.com/lapluma1219/regex4cj>

提交版本：0.3.0，具体文件以本次提交的源码为准。

原仓库：<https://github.com/rust-lang/regex>

参考版本：regex 1.13.1，提交 72d650cb0a880a01ab6dc2137c0888e8f89740f7。

文档入口：README.md；docs/api.md、api-catalog.md 和 status.md 分别说明使用

接口、公开声明及能力边界。

测试说明：docs/testing.md、docs/test-coverage.md。

验收证据：docs/validation/ 下的 latest.json、latest.log、feature-cases.json、code-coverage.json。

代码结构：port/ 为库，cli/ 为命令行入口，examples/ 为消费示例，tests/ 与 data/ 为验证材料，scripts/ 为统一脚本，oracle/ 为开发用 Rust 对照程序。

演示视频链接：【待补充】。

本次实现边界

AST/HIR 和自动机接口只实现原仓库的一部分；OnePass 当前通过适用性检查后使用锚定 PikeVM 执行，尚非独立 OnePass 引擎；Meta、Lite 仅覆盖部分能力。Rure 为仓颌包装接口，不提供 C ABI。CJD1 保存默认配置模式并重新编译，不兼容原仓库的二进制格式；不支持的配置会拒绝导出。缓存和资源限制亦不承诺与原仓库完全一致。

来源与许可

本作品基于 Rust regex 移植、适配与验证，遵循 MIT OR Apache-2.0，并保留 Unicode 数据相关许可。来源和第三方声明见 README.OpenSource、THIRD_PARTY_NOTICES.md。项目贡献包括仓颌实现、语言适配、测试与交付工程；原仓库已有算法和代码的归属按相关许可与来源声明保留。

填报承诺：本团队承诺本次参赛作品为原创开源作品，无抄袭、盗用、侵权、搬运行为，代码与文档内容真实有效，符合赛事参赛规则，特此承诺。

团队签字：闫家梁 刘孝凡

日期：2026.10.7